

MANUAL COMPLETO

Motor, juego y construcción de capítulos

Project Al.ri: Transformados

Guía de usuario, arquitectura, DSL, recetas y referencia exhaustiva de API



Edición verificada: 8 de agosto de 2026

Versión del proyecto: 1.0.0

Fuente: árbol de trabajo local y DOCUMENTACION.md

Este PDF es una exportación de consulta. Si difiere del repositorio, prevalecen DOCUMENTACION.md y el código validado.

Cómo usar este manual

El documento tiene dos recorridos. La primera parte explica cómo jugar y diagnosticar problemas. La segunda enseña cómo está construido el proyecto, cómo se escriben capítulos y cómo extender el motor. Los apéndices enumeran todas las funciones detectadas en el código actual.

Regla de autoridad

DOCUMENTACION.md conserva los manuales canónicos. Este PDF se vuelve a generar para ofrecer maquetación, capturas y una referencia de API fechada; no debe editarse a mano.

Dato verificado	Valor
Capítulos	7
Escenas	64
Líneas narrativas	921
Acciones de línea	1392
Acciones DSL	1374
Objetos complejos conservados	18
Métodos detectados en VisualNovelEngine	162
Funciones detectadas en game.js	90

Índice

Cómo usar este manual	2
Índice	3
Parte I - Manual de usuario	6
1. Arranque e instalación	6
1.1 Formas de abrir el juego	6
1.2 Primer arranque	6
2. Menú principal y navegación	6
2.1 Selector de capítulos	7
3. Controles y lectura	8
3.1 Controles superiores	8
4. Configuración, galería y accesibilidad	10
5. Minijuegos	11
5.1 Banco de QA	11
6. Resolución de problemas del jugador	12
Parte II - Manual de desarrollo	13
7. Modelo mental del runtime	13
7.1 Orden de carga	13
7.2 Flujo de una línea	13
8. Por qué se cambió la arquitectura	14
9. Intérprete de acciones y DSL	14
9.1 Sintaxis básica	14
9.2 Macro sceneStart	14
9.3 Catálogo de acciones	15
9.4 Reglas de escritura segura	16

10. Construcción de capítulos	16
10.1 Esquema actual	16
10.2 Plantilla completa	16
10.3 Orden exacto de una línea	16
10.4 Antes y ahora	17
11. Personajes, poses y assets	17
11.1 Convenciones	18
12. Renderizado, diálogo y texto	18
12.1 Typewriter	18
12.2 Capas de fondo y CG	18
13. Audio	18
14. Tiempo y pausa	18
14.1 Regla para código nuevo	19
15. Historial y navegación	19
16. Ciclo de vida de minijuegos	19
16.1 Contrato obligatorio	19
16.2 Despacho	19
17. game.js: coordinación de la aplicación	20
18. Build, validación y distribución	20
18.1 Checklist de cambio narrativo	20
Parte III - Referencia exhaustiva	21
19. Referencia de VisualNovelEngine	21
VisualNovelEngine	22
20. Referencia de módulos del motor	29
ActionInterpreter	29
AudioController	29

DOMRenderer	30
HistoryManager	30
TimeManager	30
MinigameBase	31
MediaRegistry	31
21. Referencia completa de game.js	32
Menú, Tools y navegación temprana	32
Configuración	32
Galería	33
Pausa y entrada	33
Opening y multimedia del menú	34
Capítulos y bucle principal	35
Guardado y depuración	35
22. Recetas rápidas	35
Añadir una línea con personaje	35
Cambiar de ambiente con fundido	35
Lanzar un minijuego	36
Conservar una acción compleja	36
Añadir una ruta	36
23. Glosario y fuentes internas	36
23.1 Fuentes verificadas	36

Parte I - Manual de usuario

Arranque, controles, menús, capítulos, galería, minijuegos y resolución de problemas.

1. Arranque e instalación

1.1 Formas de abrir el juego

Entorno	Procedimiento	Cuándo usarlo
Aplicación instalada	Abrir Transformados desde el acceso directo.	Jugador final.
Electron de desarrollo	npm start	Probar el escritorio con el bundle actual.
Navegador local	start.bat o npm run dev:web	Desarrollo, Tools, banco de minijuegos y tests.

1.2 Primer arranque

- Pulsa Entrar con sonido para autorizar audio en el navegador.
- Espera al opening o usa Saltar opening.
- El menú principal aparece al terminar. Si el vídeo falla, el aviso mantiene disponible el salto.
- La aplicación recuerda volumen y velocidad de texto, pero Capítulos inicia el capítulo elegido con estado narrativo limpio.

2. Menú principal y navegación



Figura 1. Menú principal. Los accesos Tools, Minijuegos y Tests sólo aparecen en desarrollo local o Electron sin empaquetar.

Opción	Resultado
Comenzar	Inicia desde el prólogo y reinicia progreso narrativo de una partida nueva.

Opción	Resultado
Capítulos	Muestra los siete capítulos detectados y arranca el elegido desde un estado limpio.
Galería	Abre arte canónico, filtros, poses, vídeo y protección de spoilers.
Configuración	Ajusta texto, sonido, ventana y ayudas opcionales.
Salir	Cierra Electron. No se muestra como cierre fiable en navegador.
Tools	Abre las herramientas gráficas locales tras comprobar el servicio del puerto 8011.
Minijuegos	Abre el catálogo de QA de minijuegos aislados.
Tests	Abre el catálogo local de pruebas de carga y rendimiento.

2.1 Selector de capítulos

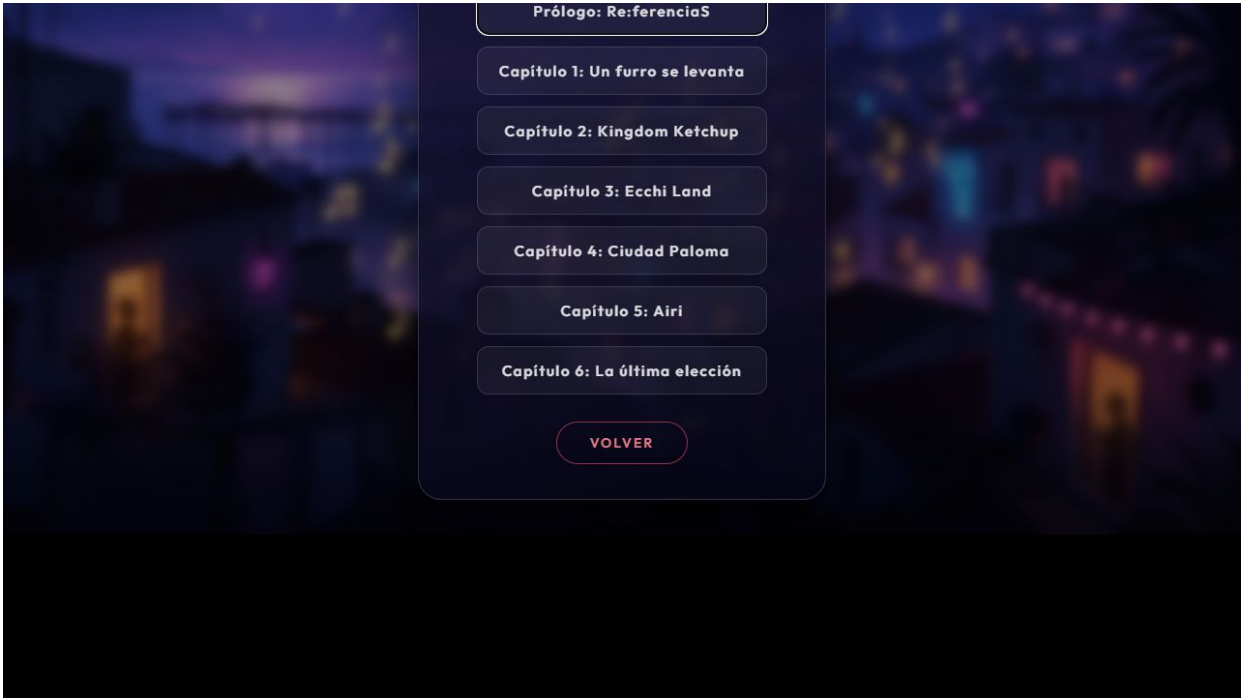


Figura 2. Selector de capítulos. No es una pantalla de partidas guardadas: carga una entrada narrativa limpia.

Archivo	Título	Escenas	Final
chapter0	Prólogo: ReferenciaS	1	No
chapter1	Capítulo 1: Un furro se levanta	3	No
chapter2	Capítulo 2: Kingdom Ketchup	18	No
chapter3	Capítulo 3: Ecchi Land	24	No
chapter4	Capítulo 4: Ciudad Paloma	7	No
chapter5	Capítulo 5: Airi	7	No
chapter6	Capítulo 6: La última elección	4	Sí

3. Controles y lectura

Entrada	Efecto
Clic / toque	Completa el texto en curso; con la línea completa, avanza.
Espacio / Enter	Equivale al clic salvo cuando una elección, vídeo o minijuego captura la entrada.
Mantener Ctrl	Acelera typewriter y líneas; nunca decide ni juega por ti.
Esc	Abre o cierra pausa; en cinemática, salta el vídeo.
H / clic derecho	Oculto o recupera todo el HUD sin avanzar el texto.
F11	Alterna pantalla completa en Windows/Electron.



Figura 3. Escena narrativa: nombre, diálogo, cursor de avance y controles superiores contextuales.

3.1 Controles superiores

Opciones pausa la partida y permite volver al menú. Escenas enumera el capítulo y distingue rutas visitadas. Retroceder restaura la fotografía de estado guardada al entrar en la escena anterior: fondo, personajes, inventario, variables y audio relevante.

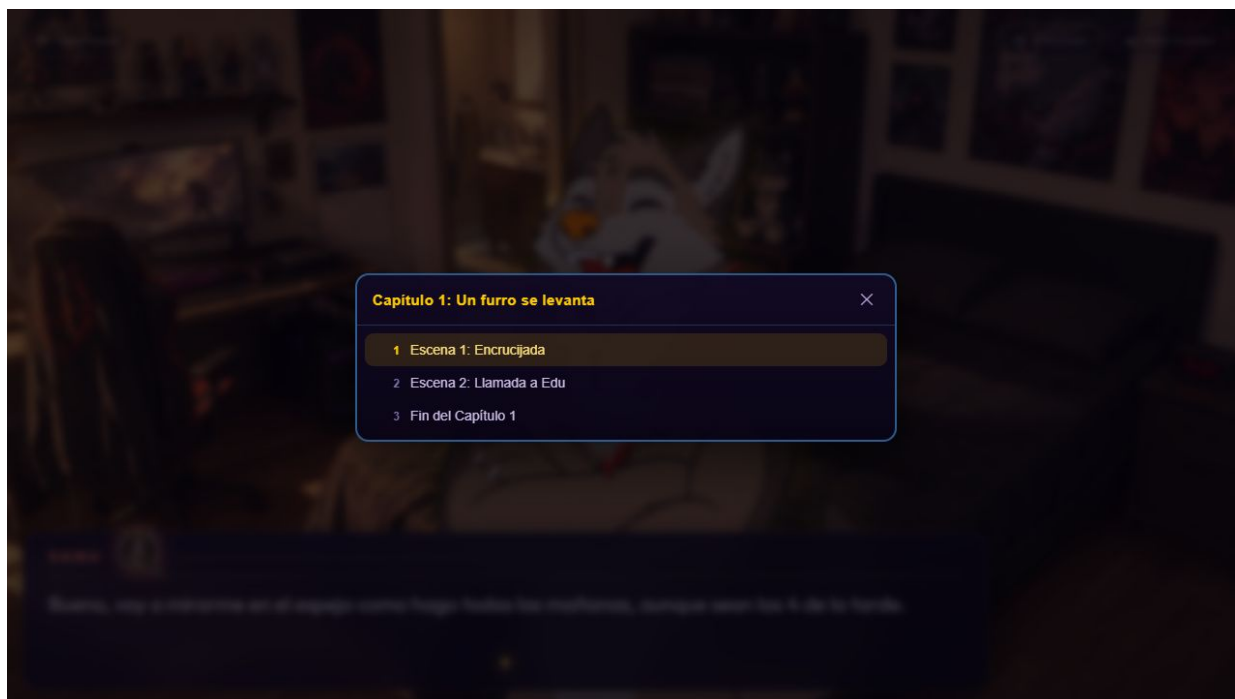


Figura 4. Menú de escenas. El salto reutiliza el bucle activo; no crea una segunda reproducción del capítulo.

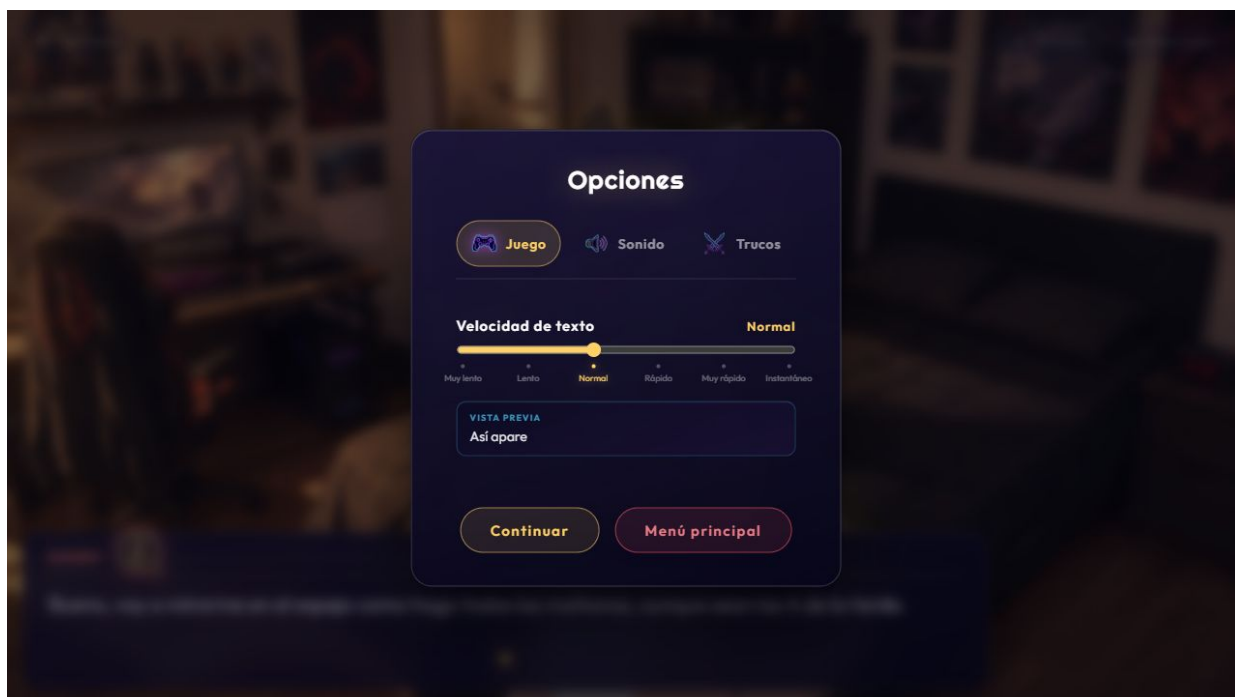


Figura 5. Pausa global. El reloj virtual, la multimedia y los contextos de audio quedan congelados.

4. Configuración, galería y accesibilidad

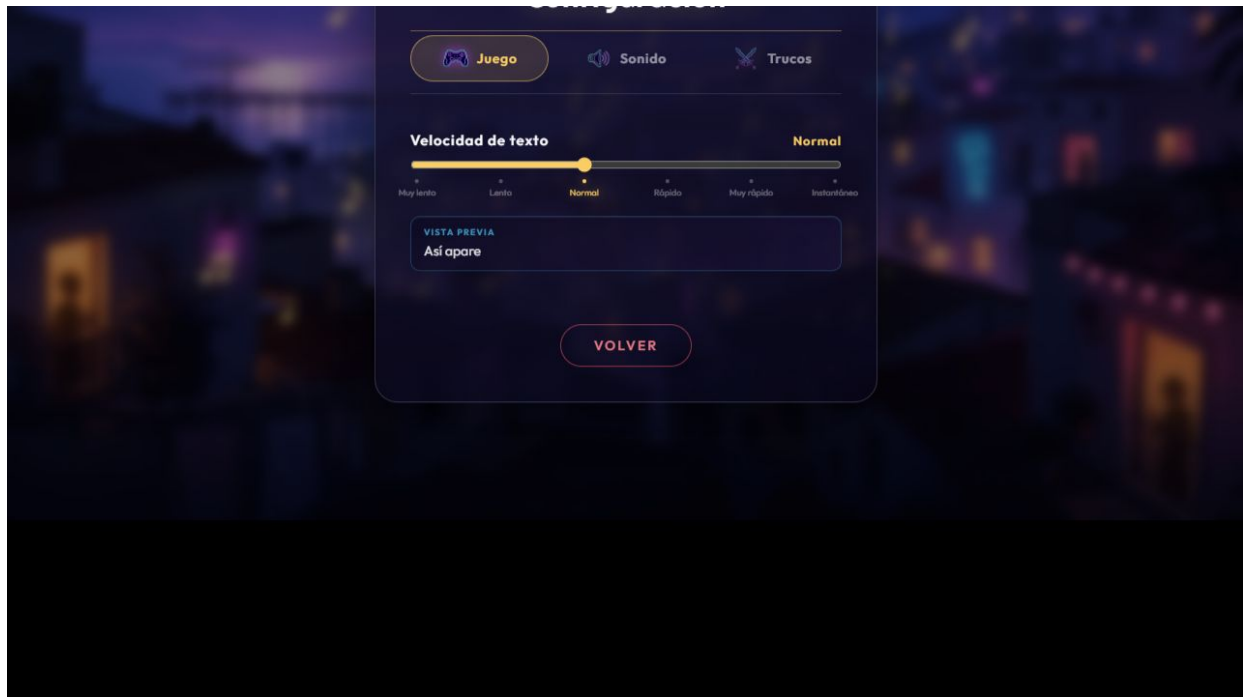


Figura 6. Configuración. Juego contiene velocidad de texto; Sonido y Trucos reúnen sus ajustes específicos.

- La vista previa de texto usa el mismo cálculo temporal que el diálogo real.
- Los volúmenes se aplican por clase: música, voz y efectos.
- La pausa conserva la música ambiental cuando corresponde, pero detiene efectos en bucle y reanuda sólo lo que ella pausó.
- El modo de ventana se sincroniza con Electron y F11.
- Ocultar HUD y los ajustes de ayuda no alteran el JSON del capítulo.

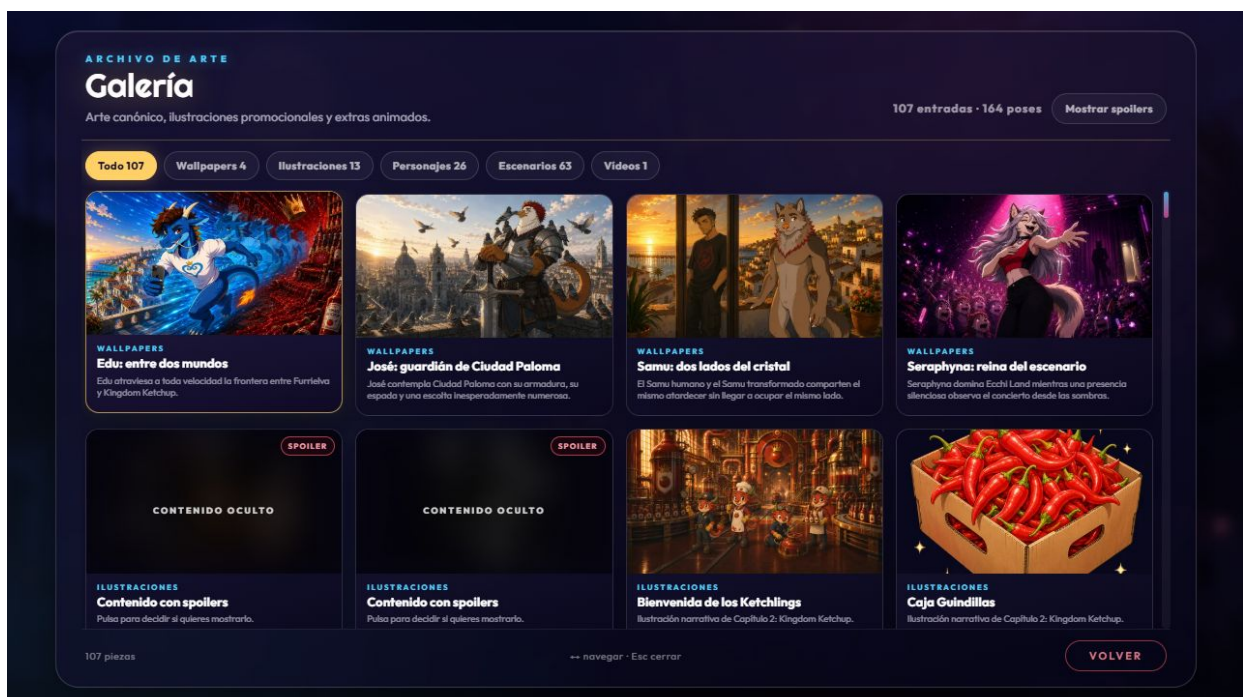


Figura 7. Galería generada desde metadatos. El contenido con spoilers permanece oculto hasta confirmación.

5. Minijuegos

Los minijuegos narrativos se lanzan desde acciones de capítulo. Durante su ejecución se mantienen disponibles Opciones, Escenas y, cuando procede, Retroceder. Abandonar la actividad cancela listeners, timers, animaciones, audio y overlays antes de continuar o volver al menú.

5.1 Banco de QA

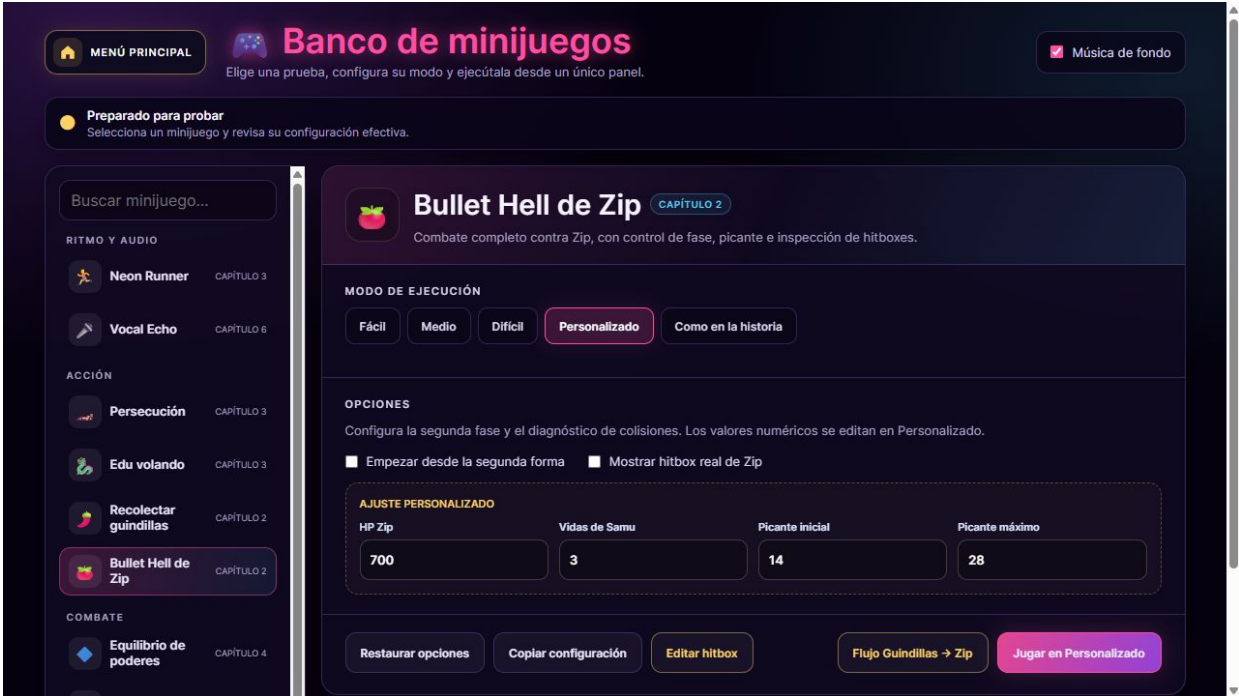


Figura 8. Banco de minijuegos. Cada ficha muestra sólo sus opciones; Personalizado habilita valores numéricos y Como en la historia lee el JSON real.

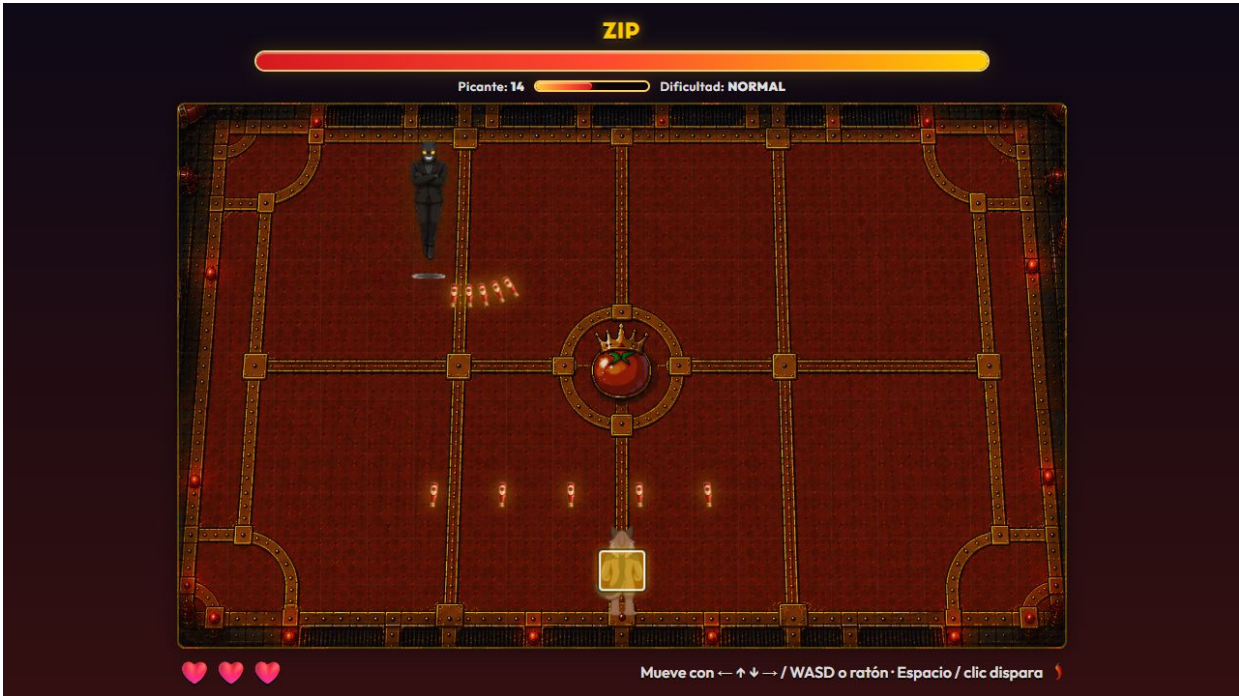


Figura 9. Bullet Hell de Zip en ejecución aislada. El banco reutiliza exactamente el motor del juego.

Grupo	Pruebas incluidas
Ritmo y audio	Neon Runner, Vocal Echo.
Acción	Persecución, Edu volando, Recolectar guindillas, Bullet Hell de Zip.

Grupo	Pruebas incluidas
Combate	Equilibrio de poderes, Battle.
Herramientas	Investigación de Furrielva, Créditos.

Salida segura

Si el menú parece no responder tras salir de un minijuego, confirma que el bundle se reconstruyó. La versión actual llama `limpiarMinijuegosActivos()` tanto al abortar como al resetear el motor.

6. Resolución de problemas del jugador

Síntoma	Comprobación y solución
No hay sonido	Pulsa Entrar con sonido; revisa Sonido y el volumen del dispositivo.
Opening bloqueado	Usa Saltar opening. Un error de vídeo no debe impedir llegar al menú.
Capítulo vacío	Ejecuta desde servidor local o Electron; abrir index.html como <code>file://</code> rompe fetch.
Botones del menú inertes	Vuelve mediante el flujo del juego. <code>setMainMenuVisible</code> restaura <code>hidden</code> , <code>inert</code> y <code>aria-hidden</code> juntos.
Minijuego sigue detrás	Reconstruye <code>engine.bundle.js</code> y verifica que <code>MinigameBase.cleanupAll</code> está incluido.
Tools no abre	Ejecuta <code>start.bat</code> o <code>npm run tools:eyes</code> y comprueba el puerto 8011.
Sprites o fondos 404	Ejecuta <code>npm run validate:content</code> y respeta rutas relativas a la raíz.

Parte II - Manual de desarrollo

Arquitectura, ejecución, DSL, capítulos, personajes, audio, pausa, historial, minijuegos, build y decisiones de diseño.

7. Modelo mental del runtime

El navegador carga un bundle clásico porque index.html y Electron consumen scripts globales. La fuente se mantiene en módulos ES, esbuild genera engine.bundle.js y después se cargan los minijuegos externos y game.js. No se usa Vite.

Capa	Responsabilidad	Archivos principales
Presentación	DOM, CSS, menú, diálogo, galería y controles.	index.html, styles.css, game.js
Orquestación	Bucle de capítulo, navegación, pausa y estado de UI.	game.js
Motor narrativo	Capítulos, acciones, personajes, audio, diálogo e historial.	engine/VisualNovelEngine.js
Servicios	Intérprete, audio, DOM, tiempo e historial.	engine/*.js
Contenido	Guion, personajes y metadatos.	chapters/, characters/, assets/metadata/
Minijuegos	Motores integrados y módulos externos con ciclo de vida común.	*-minigame.js, minigames/
Distribución	Bundle web y aplicación Electron.	engine.bundle.js, electron/, package.json

7.1 Orden de carga

```
effects.js
-> engine.bundle.js
-> battle / hitboxes / créditos / guindillas / Ketchup / runas
-> juice.js
-> game.js
```

7.2 Flujo de una línea

```
playGame()
-> engine.nextLine()
-> comprobar showIf y consecuencias
-> ejecutar actions en orden
-> displayDialog(line)
-> ejecutar afterActions
-> displayChoices() si existen
-> waitForClick()
-> repetir o endGame()
```

8. Por qué se cambió la arquitectura

Cambio	Problema anterior	Razón y efecto actual
DSL JSON	Cada acción simple repetía type y nombres de propiedades.	Reduce ruido y mantiene objetos para estructuras complejas.
ActionInterpreter	El switch de acciones estaba mezclado con todo el motor.	Centraliza parseo, alias, macros y despacho.
TimeManager	game.js parcheaba timers, Date.now, performance.now y RAF.	La pausa se instala una vez y tiene pruebas deterministas.
Controladores	Audio, DOM e historial dependían del monolito sin una frontera explícita.	Ofrecen puntos de entrada pequeños y comprobables.
Bundle esbuild	Los navegadores clásicos y Electron necesitaban globals.	La fuente usa import/export sin cambiar la forma de arrancar.
MinigameBase	Salir eliminaba el overlay, pero podían sobrevivir listeners y timers.	Registra recursos e instancias y permite cleanupAll.
Variables CSS	Colores y z-index mágicos se repetían.	Centraliza identidad visual y capas globales.

Compatibilidad primero

La migración no obliga a reescribir todo de una vez: parseAction acepta objetos heredados, serializeAction sólo convierte cuando el round-trip es exacto y engine.bundle.js conserva los globals que ya espera el HTML.

9. Intérprete de acciones y DSL

ActionInterpreter.normalize acepta un objeto, una cadena o una macro. execute procesa arrays expandidos y executeObject despacha el tipo normalizado. Los escalares key=value reconocen números, true, false y null; los textos con espacios deben ir entre comillas.

9.1 Sintaxis básica

```
show nexo right happy
background assets/images/backgrounds/chapter0/habitacion_neon.webp
sound assets/audio/music/chapter0/nexo.mp3 volume=1 loop=true id=nexo fadeIn=600
fade to black duration=350
goto "Escena 4: Mercaguasa"
minigame battle enemy=amalgama enemyHp=840 retryOnDefeat=true
```

9.2 Macro sceneStart

```
sceneStart assets/images/backgrounds/chapter2/fabrica.webp duration=400

# Expansión conceptual:
fade to black duration=400
background assets/images/backgrounds/chapter2/fabrica.webp
fade from black duration=400
```


9.3 Catálogo de acciones

Comando / tipo	Campos frecuentes	Efecto
background / setBackground	ruta, cut, duration	Cambia el fondo y cancela cruces pendientes incompatibles.
clearBackground / removeBackground	-	Vacía fondo y retira personajes al cambiar de escenario.
show / showCharacter	character position pose; enter, flipped, scale, offsetY	Muestra un personaje.
hide / hideCharacter	character o position; exit	Oculto sin olvidar necesariamente la composición.
remove / removeCharacter / quitarPersonaje	character o position; exit	Retira el sprite y su estado de escenario.
pose / setPose	character position pose	Cambia la pose actual.
animateCharacter / characterAnimation / poseSequence	poses[], frameMs, loop, untilAdvance	Anima una secuencia; suele conservarse como objeto.
stopCharacterAnimation / stopPoseSequence	character, position	Detiene la secuencia de pose.
characterGlitch / glitchCharacter	character, position, duration	Aplica glitch local temporal.
characterFullGlitch / fullCharacterGlitch	character, position, duration	Aplica glitch completo.
characterGlitchUntilAdvance / glitchUntilAdvance	character, position	Mantiene glitch hasta avanzar.
characterAnimeFall / animeFall	character, position, opciones	Dispara caída estilizada.
dialogOff / hideDialog / hideText / ocultarTexto	-	Oculto el cuadro de texto.
setVariable	name/value o key/value	Escribe estado narrativo.
item / giveItem / addItem	item	Añade inventario.
rescue	character	Registra rescate manteniendo orden.
delay / setDelay / addDelay	value	Establece o suma presión/retraso de historia.
sound / playSound	path; id, volume, loop, fadeIn	Reproduce audio controlable.
stopSound	id, fadeOut	Detiene un audio concreto.
stopAllSounds	-	Detiene y libera toda la mezcla.
pauseSound / resumeSound	id	Pausa o reanuda una instancia.
volume / setVolume	id volume	Cambia volumen base.
wait	ms	Espera tiempo del reloj pausable.
setTextDuration / textDuration	duration	Fija duración de la línea.
click / waitForClick / waitClick / esperarClick	-	Exige un gesto antes de continuar acciones.
minigame	game y opciones	Despacha una actividad y espera su resultado.
goto / goToScene	título o índice	Solicita salto de escena.
nextChapter / setNextChapter	id	Define la siguiente rama de capítulo.
video / playVideo / cutscene	path y fundidos	Reproduce cinemática saltable.
shake / screenShake	intensity duration	Agita el escenario.
flash	color duration	Destello de pantalla.
grade / colorGrade / tinte	value duration	Aplica corrección de color.
vignette / vigneta	value duration	Controla viñeta visual.
fade	to/from color, duration	Fundido de escena.
bgPan	zoomFrom zoomTo duration	Paneo o zoom del fondo.
cg / showCG	path duration	Muestra ilustración a pantalla.
hideCG	duration	Retira la ilustración.
sfx	name on	Activa un efecto Juice con nombre.

9.4 Reglas de escritura segura

- Usa DSL para acciones planas con escalares.
- Conserva objeto si hay arrays, mapas anidados, startItems o secuencias de poses.
- Pon entre comillas títulos de escena y cualquier valor con espacios, comillas o signo igual.
- Las herramientas que inspeccionen capítulos deben llamar `engine.actionInterpreter.normalize(action)` antes de leer `action.game` o `action.type`.
- Ejecuta `npm run validate:content` y las pruebas del motor después de migrar.

10. Construcción de capítulos

Cada archivo `chapters/chapterN.json` declara título, escenas y líneas. La línea es la unidad ejecutable: puede tener acciones previas, hablante, texto, expresión, condiciones, decisiones y acciones posteriores.

10.1 Esquema actual

Nivel	Campos observados	Responsabilidad
Capítulo	title, scenes, isFinal	Identidad, secuencia y terminación.
Escena	title, lines	Unidad de navegación e historial.
Línea	actions, character, text, pose, position, emotion, speakingAs, textSpeed, textAnimation, showIf, choices, afterActions, type	Diálogo y comportamiento ejecutable.
Elección	text, nextScene, nextChapter	Etiqueta y destino narrativo.

10.2 Plantilla completa

```
{
  "title": "Capítulo X: Título",
  "scenes": [
    {
      "title": "Escena 1: Llegada",
      "lines": [
        {
          "actions": [
            "sceneStart assets/images/backgrounds/chapterX/llegada.webp",
            "show samu left surprised",
            "sound assets/audio/music/chapterX/tema.mp3 id=bg_music loop=true"
          ],
          "character": "Samu",
          "text": "Ya estamos aquí.",
          "emotion": "surprise",
          "afterActions": ["pose samu left determined"]
        },
        {
          "character": "3C",
          "text": "Elige con cuidado.",
          "choices": [
            { "text": "Entrar", "nextScene": "Escena 2: Interior" },
            { "text": "Esperar", "nextChapter": "chapter_rama" }
          ]
        }
      ]
    }
  ]
}
```

10.3 Orden exacto de una línea

- Evaluar `showIf` y condiciones de consecuencia.
- Precargar los assets detectables de la línea.
- Ejecutar actions en orden y esperar cada acción asíncrona.
- Mostrar texto, retrato, pose y emoción.

- Ejecutar afterActions cuando corresponda.
- Mostrar choices y aplicar nextScene o nextChapter.
- Marcar isWaitingForInput para que game.js espere clic sin crear otro bucle.

10.4 Antes y ahora

Métrica	Antes (HEAD)	Ahora
Acciones totales	1392	1392
Cadenas DSL	1374	1374
Objetos	18	18
Capítulos / escenas / líneas	7 / 64 / 921	7 / 64 / 921

Formato anterior

```
"actions": [
  {
    "type": "showCharacter",
    "character": "samu",
    "position": "left",
    "pose": "surprised"
  }
]
```

Formato actual equivalente

```
"actions": ["show samu left surprised"]
```

Objeto complejo que debe conservarse

```
{
  "type": "animateCharacter",
  "character": "nexo",
  "position": "right",
  "poses": ["happy", "neutral", "happy"],
  "frameMs": 220,
  "loop": true,
  "untilAdvance": true
}
```

Por qué no convertirlo todo

Una cadena es ideal cuando la acción es plana. Arrays y objetos anidados perderían estructura o exigirían una sintaxis difícil de leer; por eso el migrador sólo escribe si `parseAction(serializedAction(objeto))` reproduce exactamente el original.

11. Personajes, poses y assets

`characters/<id>.json` contiene name, color, poses, defaultPose y animaciones. El motor normaliza el identificador, carga la ficha una vez y aplica manifiestos opcionales de limpieza, capas de ojos y parpadeos.

```
{
  "name": "Nexo",
  "color": "#1b8587",
  "poses": {
    "neutral": "assets/images/characters/others/nexo.webp",
    "happy": "assets/images/characters/others/nexo.webp"
  },
  "defaultPose": "neutral",
  "animations": {
    "neutral": {"frames": [{"src": "...blink_closed.webp", "duration": 110}], "loop": true}
  }
}
```

11.1 Convenciones

- El id del archivo es la clave técnica; name es la etiqueta visible.
- Cada pose referenciada por un capítulo debe existir o tener fallback explícito.
- Las rutas son relativas a la raíz del proyecto y deben usar barras /.
- Los masters y originales se conservan en el árbol workbench con la misma ruta relativa; QA y descartes no se guardan allí.
- Narradora es 3C y Nexa es su auxiliar de continuidad; ePod no forma parte del canon activo.

12. Renderizado, diálogo y texto

DOMRenderer ofrece una frontera pequeña para buscar nodos y añadir o retirar contenido. VisualNovelEngine mantiene todavía la composición detallada: dos fondos para crossfade, tres slots de personajes, CG, capas de ojos, cuadro de diálogo y efectos Juice.

12.1 Typewriter

- splitTextGraphemes evita cortar emojis o grafemas combinados.
- getTextCharacterDelay añade pausas por puntuación y aplica la velocidad elegida.
- calculateTextTiming produce duración visible y total.
- setLineTextDuration comparte la duración con efectos sincronizados.
- Un clic durante la escritura completa; el siguiente avanza.
- speakingAs permite que el nombre visible y el sprite enfocado no sean la misma identidad.

12.2 Capas de fondo y CG

setBackground alterna background y background-b. clearBackground cancela el timer de intercambio para impedir que un crossfade antiguo restaure una imagen retirada. showCG e hideCG usan una capa independiente para ilustraciones narrativas.

13. Audio

Operación	Uso
playSound(path, options)	Reproduce música, voz o efecto con id, loop, volumen y fadeln.
stopSound(id, fadeOut)	Detiene una instancia o id con fundido opcional.
pauseSound / resumeSound	Control explícito sin perder posición.
setVolume	Modifica el volumen base antes del factor global.
stopAllSounds	Cancela fades, pausa, libera src y vacía pools/registro.
AudioController	Fachada pequeña usada por el intérprete y pruebas.

Por qué se libera src

Un audio pausado puede seguir reteniendo una conexión de streaming. stopAllSounds libera el recurso para no agotar el límite de conexiones del navegador durante sesiones largas.

14. Tiempo y pausa

TimeManager instala versiones pausables de setTimeout, setInterval, requestAnimationFrame, Date.now y performance.now. Al pausar, guarda el tiempo restante; al reanudar, arma cada trabajo una sola vez. PausableMediaRegistry mantiene audios y contextos WebAudio conocidos por el juego.

```

setGamePaused(true)
-> pausar multimedia y contextos no musicales
-> gamePauseClock.setPaused(true)
-> emitir illo:pausechange

setGamePaused(false)
-> rearmar timers y RAF pendientes
-> reanudar sólo los medios pausados por el menú

```

14.1 Regla para código nuevo

Usa los timers globales después de instalar engine.bundle.js: quedan interceptados por TimeManager. Dentro de MinigameBase usa timeout() y animationFrame() para que, además de pausables, sean cancelables al abandonar la actividad.

15. Historial y navegación

Cada entrada de escena conserva cursores, variables, inventario, personajes, fondo y otros datos necesarios para reconstruir la presentación. HistoryManager delega captureStage, restoreStage, rewind, jump y clear al motor sin duplicar el formato de la fotografía.

Ruta	Qué ocurre
Retroceder	Restaura la fotografía anterior y reproduce la escena desde su línea 0.
Escena visitada	Recupera la fotografía guardada de esa entrada.
Escena no visitada	Salta conservando el progreso actual y deja que la escena registre su entrada.
Salto de línea de depuración	Limpia escenario y cambia cursor dentro del mismo playGame.
Salida al menú	Desbloquea cualquier espera, cancela actividad y resetea el motor.

16. Ciclo de vida de minijuegos

```

class MiMinijuego extends window.MinigameBase {
  start() {
    this.state = 'running';
    return new Promise(resolve => {
      this.resolve = resolve;
      const overlay = this.attachOverlay(document.createElement('div'));
      this.listen(document, 'keydown', event => this.onKey(event));
      this.timeout(() => this.finish(false), 30000);
      this.animationFrame(time => this.tick(time));
      document.querySelector('#game-container').appendChild(overlay);
    });
  }
}

```

16.1 Contrato obligatorio

- start devuelve una Promise que termina en victoria, derrota o cancelación.
- pause y resume cambian estado sin crear bucles adicionales.
- listen, timeout, animationFrame y own registran recursos que cleanup debe retirar.
- attachOverlay registra la instancia y observa la retirada externa del nodo.
- cancel elimina overlay, limpia recursos y resuelve false si hay una promesa pendiente.
- cleanupAll se usa antes de reset y al abortar desde Opciones o Escenas.

16.2 Despacho

La acción minigame llega a VisualNovelEngine.playMinigame. El switch llama al motor correspondiente y compite contra una Promise de cancelación. La cancelación rechaza con la marca minijuegoCancelado, limpia sin ruido y permite que el bucle atienda salida, salto o retroceso.

17. game.js: coordinación de la aplicación

game.js no interpreta acciones. Su responsabilidad es unir DOM, menú, galería, configuración, pausa y el bucle de juego con una instancia de VisualNovelEngine. Mantiene banderas de navegación para que exista un solo playGame activo.

Subsistema	Funciones clave
Menú	setMainMenuVisible, showMenuMedia, stopMenuMedia, startNewGame.
Capítulos	ensureAvailableChapters, showChapterSelector, startChapterFromSelector.
Bucle	playChapter, playGame, waitForClick, endGame.
Navegación	desbloquearBucle, abrirMenuEscenas, volverAlMenuPrincipal.
Pausa	setGamePaused, abrirMenuPausa, cerrarMenuPausa.
Galería	loadGalleryManifest, renderGalleryPanel, closeGalleryPanel.
Arranque	setupStartupSequence, startStartupOpening, showMainMenuAfterOpening.
Depuración	initDebugMode, toggleDebugPanel, goToLine mediante engine.

18. Build, validación y distribución

Comando	Qué comprueba o produce
npm run build:engine	Genera engine.bundle.js con esbuild; no usa Vite.
npm run validate:content	Capítulos, acciones DSL, personajes, poses y referencias de assets.
npm run check:modules	Formato de módulos nuevos, scripts de DSL y tests.
npm run test:engine	Intérprete, controladores, MinigameBase y TimeManager.
npm run test:repeat -- 10	Repite bundle, contenido y motor para detectar inestabilidad.
npm run validate:workbench	Espejo de masters y referencias de workbench.
npm run audit:assets	Detecta assets candidatos a optimización.
npm run dist:dir	Empaqueta Electron sin crear instalador.
npm run dist	Construye el instalador de Windows.
python scripts/generate_engine_manual_pdf.py	Regenera este PDF desde el estado actual.

18.1 Checklist de cambio narrativo

- Editar chapters/chapterN.json y fichas de personaje necesarias.
- Usar DSL sólo para acciones planas; mantener objetos complejos.
- Ejecutar validate:content y test:engine.
- Abrir el capítulo desde Capítulos y probar rutas, elecciones, audio y minijuegos.
- Revisar salida a Opciones, Escenas y menú durante esperas largas.
- Actualizar Manual de usuario si cambia comportamiento visible y Manual de desarrollo si cambia esquema, API o procedimiento.
- Regenerar el bundle y, si se entrega documentación, este PDF.

Parte III - Referencia exhaustiva

Inventarios generados desde las firmas y comentarios del código. Las líneas corresponden al árbol de trabajo verificado.

19. Referencia de VisualNovelEngine

Se detectaron 162 métodos de VisualNovelEngine. También se incluye MinigameLifeDisplay, definido en el mismo archivo. async indica Promise; interna señala nombres con prefijo _.

VisualNovelEngine

Función y firma	Ubicación	Responsabilidad
constructor()	engine/VisualNovelEngine.js:45	Inicializa la instancia y sus dependencias internas.
_getBackgroundElement() [interna]	engine/VisualNovelEngine.js:123	Implementa la operación get background element del módulo.
addItem(name)	engine/VisualNovelEngine.js:132	Implementa la operación add item del módulo.
hasItem(name)	engine/VisualNovelEngine.js:141	Comprueba si existe item dentro del flujo correspondiente.
setFastForward(active)	engine/VisualNovelEngine.js:150	Establece fast forward dentro del flujo correspondiente.
setTextPaused(paused)	engine/VisualNovelEngine.js:163	Establece text paused dentro del flujo correspondiente.
async fetchJson(path, options = {async})	engine/VisualNovelEngine.js:178	Implementa la operación fetch json del módulo.
async loadChapter(chapterName) [async]	engine/VisualNovelEngine.js:223	Lee y valida el JSON del capítulo, reinicia cursores y prepara la primera escena.
_extractLinePrefetchTargets(line) [interna]	engine/VisualNovelEngine.js:271	Implementa la operación extract line prefetch targets del módulo.
async _prefetchLineAssetsFromTargets(targets) [async, interna]	engine/VisualNovelEngine.js:336	Implementa la operación prefetch line assets from targets del módulo.
prefetchLineAssets(chapter, sceneIndex = 0, lineIndex = 0)	engine/VisualNovelEngine.js:350	Implementa la operación prefetch line assets del módulo.
prefetchNextLineAssets()	engine/VisualNovelEngine.js:388	Implementa la operación prefetch next line assets del módulo.
async loadCharacter(characterName) [async]	engine/VisualNovelEngine.js:411	Carga la ficha JSON de personaje y aplica capas limpias, parpadeos y animaciones disponibles.
async loadBlinkIntermediateManifest() [async]	engine/VisualNovelEngine.js:457	Carga blink intermediate manifest dentro del flujo correspondiente.
async loadWhiteHaloManifest() [async]	engine/VisualNovelEngine.js:470	Carga white halo manifest dentro del flujo correspondiente.
applyWhiteHaloCopies(characterKey, character, manifest)	engine/VisualNovelEngine.js:483	Aplica white halo copies dentro del flujo correspondiente.
applyWhiteHaloAnimationFrames(characterKey, character, manifest)	engine/VisualNovelEngine.js:494	Aplica white halo animation frames dentro del flujo correspondiente.
async loadLayerBlinkManifest() [async]	engine/VisualNovelEngine.js:513	Carga layer blink manifest dentro del flujo correspondiente.
applyLayerBlinkFrames(characterKey, character, manifest)	engine/VisualNovelEngine.js:560	Aplica layer blink frames dentro del flujo correspondiente.
getLayerBlinkConfig(characterKey, pose)	engine/VisualNovelEngine.js:569	Obtiene layer blink config dentro del flujo correspondiente.
applyBlinkIntermediateFrames(characterKey, character, manifest)	engine/VisualNovelEngine.js:573	Aplica blink intermediate frames dentro del flujo correspondiente.
getCurrentScene()	engine/VisualNovelEngine.js:598	Obtiene current scene dentro del flujo correspondiente.
getCurrentLine()	engine/VisualNovelEngine.js:609	Obtiene current line dentro del flujo correspondiente.
resolveConsequenceLine(line)	engine/VisualNovelEngine.js:627	Resuelve consequence line dentro del flujo correspondiente.
async executeAction(action) [async]	engine/VisualNovelEngine.js:656	Normaliza una acción objeto o DSL y la delega al intérprete.
playVideo(action = {})	engine/VisualNovelEngine.js:667	Reproduce o ejecuta video dentro del flujo correspondiente.
clearBackground()	engine/VisualNovelEngine.js:849	Salta a una escena por título (o índice) dentro del capítulo actual. Marca un salto pendiente para que nextLine detenga el procesamiento normal de la línea y no avance automáticamente. Vaciar el fondo del todo (lo usa el final del compí en

Función y firma	Ubicación	Responsabilidad
jumpToScene(target)	engine/VisualNovelEngine.js:870	Salta a to scene dentro del flujo correspondiente.
rescueCharacter(name)	engine/VisualNovelEngine.js:892	Registrar un personaje como rescatado (mantiene el orden de rescate)
async playMinigame(action) [async]	engine/VisualNovelEngine.js:898	Despacha el minijuego solicitado y coordina cancelación, reintento, audio y limpieza global.
hayMinijuegoAbierto()	engine/VisualNovelEngine.js:1003	¿Hay un minijuego en marcha que se pueda abortar? Lo usan los botones de arriba, que ahora se ven también durante los minijuegos.
abortarMinijuego()	engine/VisualNovelEngine.js:1007	Cancela minijuego dentro del flujo correspondiente.
limpiarMinijuegosActivos()	engine/VisualNovelEngine.js:1016	Quitar solo el overlay no basta: un minijuego externo puede conservar RAF, timers, audio o listeners. La base cancela primero esos recursos; los integrados detectan después que su overlay ya no está conectado.
playFurriervaExploreMinigame(options = { ... })	engine/VisualNovelEngine.js:1029	Investigación narrativa de Furrierva. Furry Maps presenta las zonas como áreas reales del mapa: Samu las introduce, el jugador confirma cada ruta y un Samu diminuto recorre el trayecto desde su última ubicación. Solo el primer desplazamiento.
async playCreditsMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:1582	Reproduce o ejecuta credits minigame dentro del flujo correspondiente.
async playBattleMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:1599	Reproduce o ejecuta battle minigame dentro del flujo correspondiente.
async playRuneChannelingMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:1651	Reproduce o ejecuta rune channeling minigame dentro del flujo correspondiente.
async playChiliHarvestMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:1676	Reproduce o ejecuta chili harvest minigame dentro del flujo correspondiente.
runChiliHarvestRound(options = { ... })	engine/VisualNovelEngine.js:1689	Ejecuta chili harvest round dentro del flujo correspondiente.
runLegacyChiliHarvestRound(options = { ... })	engine/VisualNovelEngine.js:1697	Ejecuta legacy chili harvest round dentro del flujo correspondiente.
async playKetchupBossMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:1895	Bullet hell de José Manuel, conectado a la reserva de guindillas obtenida en la ronda anterior y al posible regalo de Neit.
async playKetchupMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:1923	Alias para herramientas y escenas antiguas que invocaban este nombre.
runKetchupRound(options = { ... })	engine/VisualNovelEngine.js:1929	Implementación histórica del recolector, conservada temporalmente para compatibilidad con pruebas antiguas; la historia usa chiliHarvest.
showMinigameRetry(message = '¡No ha salido bien!')	engine/VisualNovelEngine.js:2202	Muestra minigame retry dentro del flujo correspondiente.
hayRetryAbierto()	engine/VisualNovelEngine.js:2248	¿Hay una pantalla de reintento esperando? La usan los botones de escenas y de retroceder para saber si tienen que sacarnos de ahí.
abortarRetry()	engine/VisualNovelEngine.js:2252	Cancela retry dentro del flujo correspondiente.
async playGatosMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:2260	Minijuego: Micaela Michis (El Jarrón). Estilo Pac-Man: Samu se mueve libremente por el campo y debe SOBREVIVIR huyendo de los gatos que le persiguen durante un tiempo (por defecto 60 s). Si un gato lo alcanza, pierde y puede reintentar.
runGatosRound(options = { ... })	engine/VisualNovelEngine.js:2302	Una ronda del minijuego de gatos, estilo Pac-Man por REJILLA. Samu y los gatos recorren las calles del laberinto (giran en las intersecciones). Los gatos persiguen a Samu con búsqueda de camino (BFS) por las calles. Resuelve con true (sobre
async playEcchiMinigame(options = { ... }) [async]	engine/VisualNovelEngine.js:2644	Reproduce o ejecuta ecchi minigame dentro del flujo correspondiente.
runEcchiRound(options = { ... })	engine/VisualNovelEngine.js:2663	Ejecuta ecchi round dentro del flujo correspondiente.

Función y firma	Ubicación	Responsabilidad
async playPalomaMinigame(options = [async])	engine/VisualNovelEngine.js:2797	Reproduce o ejecuta paloma minigame dentro del flujo correspondiente.
runPalomaRound(options = [async])	engine/VisualNovelEngine.js:2827	Ejecuta paloma round dentro del flujo correspondiente.
async playRunaMinigame(options = [async])	engine/VisualNovelEngine.js:2955	Minijuego: memoria de runas (estilo Simon).
runRunaRound(options = [async])	engine/VisualNovelEngine.js:2970	Una partida de memoria: repite la secuencia de runas que se ilumina. La secuencia crece cada nivel hasta completar `rounds`. Resuelve true/false.
async playVocalEchoMinigame(options = [async])	engine/VisualNovelEngine.js:3109	===== Minijuego: "Vocal Echo / Dúo" (Simon musical) - capítulo de Tony. Tony canta una secuencia de notas (pads que se iluminan y suenan con Web Audio); el jugador la repite. Arranca co
runVocalEchoRound(options = [async])	engine/VisualNovelEngine.js:3121	Ejecuta vocal echo round dentro del flujo correspondiente.
async playRhythmMinigame(options = [async])	engine/VisualNovelEngine.js:3299	===== Minijuego: "Neon Runner" (ritmo tipo DDR) - capítulo de Tony. Caen notas por varios carriles; hay que pulsar la tecla del carril (o clicar el carril) cuando la nota cruza la línea
runRhythmRound(options = [async])	engine/VisualNovelEngine.js:3317	Ejecuta rhythm round dentro del flujo correspondiente.
async playChaseMinigame(options = [async])	engine/VisualNovelEngine.js:4019	===== Minijuego "chase" (persecución en coche): esquivar obstáculos y a los memes en moto durante una distancia. Reutiliza el motor side-scroller. =====
async playEduVueloMinigame(options = [async])	engine/VisualNovelEngine.js:4117	Reproduce o ejecuta edu vuelo minigame dentro del flujo correspondiente.
async runEduFlight(cfg = [async])	engine/VisualNovelEngine.js:4160	Ejecuta edu flight dentro del flujo correspondiente.
async runSideScroller(cfg) [async]	engine/VisualNovelEngine.js:5148	Motor común de los side-scrollers. Devuelve Promise<boolean> (ganado).
ensureSceneLayers()	engine/VisualNovelEngine.js:6113	Capas de dirección de escena (crossfade de fondo, fundido a negro y CG). Se crean perezosamente colgando de #game-container (regla del stage escalado).
setBackground(imagePath, opts = [async])	engine/VisualNovelEngine.js:6152	Cambia el fondo con CROSSFADE suave (400 ms) usando una segunda capa. Con { cut: true } se comporta como antes (corte seco intencionado).
fadeScene(action = [async])	engine/VisualNovelEngine.js:6209	Implementa la operación fade scene del módulo.
bgPan(action = [async])	engine/VisualNovelEngine.js:6234	Ken Burns del fondo: zoom/paneo lento. Se resetea al cambiar de fondo.
async showCG(path, duration = 600, opts = [async])	engine/VisualNovelEngine.js:6264	Lámina CG a pantalla (por encima de personajes, por debajo del diálogo). Muestra una ilustración a pantalla completa. Con `size` se puede mostrar un OBJETO en vez de una escena: las CGs normales son apaisadas y llenan la pantalla (cover), p
hideCG(duration = 500)	engine/VisualNovelEngine.js:6284	Ocultar cg dentro del flujo correspondiente.
cacheBustAsset(path)	engine/VisualNovelEngine.js:6299	Rompe la caché UNA VEZ POR CARGA DE PÁGINA, no en cada llamada. Antes esto devolvía `v=\${Date.now()}`, así que cada cambio de fotograma de un sprite generaba una URL nueva y el navegador se volvía a descargar el PNG entero: el hueco se que
preloadImages(paths)	engine/VisualNovelEngine.js:6318	Precarga una lista de imágenes y conserva tanto la promesa como el Image. Si se repite un minijuego o dos motores comparten un sprite, se reutiliza exactamente la misma descarga y la misma decodificación en memoria.
_normalizeLookupKey(value) [interna]	engine/VisualNovelEngine.js:6355	Implementa la operación normalize lookup key del módulo.

Función y firma	Ubicación	Responsabilidad
getCharacterKey(characterName)	engine/VisualNovelEngine.js:6362	Obtiene character key dentro del flujo correspondiente.
getCharacterPoseImage(character, pose)	engine/VisualNovelEngine.js:6391	Obtiene character pose image dentro del flujo correspondiente.
applyCharacterPoseImage(charElement, poseImage)	engine/VisualNovelEngine.js:6410	Un cambio de pose es un reemplazo limpio. La versión anterior creaba una segunda capa con el sprite viejo; si la clase de animación se retiraba antes de tiempo, esa copia quedaba visible detrás del personaje nuevo. Las animaciones reales se
applyCharacterAnimationFrame(charElement, framePath)	engine/VisualNovelEngine.js:6425	Aplica character animation frame dentro del flujo correspondiente.
hideCharacterEyeLayer(charElement)	engine/VisualNovelEngine.js:6432	Ocultar character eye layer dentro del flujo correspondiente.
applyCharacterEyeLayerFrame(charElement, config, frame)	engine/VisualNovelEngine.js:6441	Aplica character eye layer frame dentro del flujo correspondiente.
animationFramePath(character, frame)	engine/VisualNovelEngine.js:6475	Implementa la operación animation frame path del módulo.
animationDelay(value, fallback = 2400)	engine/VisualNovelEngine.js:6481	Implementa la operación animation delay del módulo.
startCharacterFrameAnimation(characterKey, position, pose)	engine/VisualNovelEngine.js:6494	Reproduce fotogramas pertenecientes a UNA misma pose. Cuando existen recortes oculares, la base permanece inmóvil y sólo cambia el parche de ojos (medio/cerrado/medio); el resto conserva el fallback de sprite completo.
stopCharacterFrameAnimation(position, restoreBase = true)	engine/VisualNovelEngine.js:6593	Detiene character frame animation dentro del flujo correspondiente.
humanDialogPortrait(characterKey)	engine/VisualNovelEngine.js:6613	Edu, Tony y José conservan su retrato humano en el cursor hasta el instante narrativo en que el juego revela su forma transformada. Los umbrales por escena/línea permiten también saltar desde el menú de escenas sin depender de una variable
isFurryIdentityRevealed(characterKey)	engine/VisualNovelEngine.js:6622	Comprueba si está furry identity revealed dentro del flujo correspondiente.
async showCharacter(characterName, position = 'left', pose = 'neutral', flipped = false, enter = null, offsetY = null, scale = null,) [async]	engine/VisualNovelEngine.js:6638	Coloca un sprite con pose, entrada, escala, orientación y desplazamiento vertical.
resolveVerticalOffset(offsetY)	engine/VisualNovelEngine.js:6736	El guión manda el encuadre de cada aparición: 'offsetY' baja (o sube, en negativo) al personaje esa fracción de su propia altura, para escenas donde la estatura es parte de la narración (los Ketchlings miden cuarenta centímetros y solo asom
resolveScale(scale)	engine/VisualNovelEngine.js:6743	'scale' en showCharacter dibuja al personaje más grande o más pequeño sin tocar su sprite (José un 18% más grande, Airi de niña un 30% más pequeña).
layoutCharacters()	engine/VisualNovelEngine.js:6754	Reparte a los personajes ACTIVOS en franjas horizontales iguales. IMPORTANTE: el ancho es FIJO (no depende de cuántos haya), así con 3 personajes NO se encogen; solo se separan. Como los sprites son verticales y usan background-size: contain
setPose(characterName, position, pose = 'neutral', options =	engine/VisualNovelEngine.js:6775	Establece pose dentro del flujo correspondiente.
startCharacterPoseAnimation(action =	engine/VisualNovelEngine.js:6803	Secuencia declarativa de poses. Permite pequeños acting loops usando los sprites ya disponibles: parpadeos, respiración, nervios o sacudidas. Por defecto vive hasta que el jugador avanza la línea; también admite una secuencia finita con loop
stopCharacterPoseAnimation(characterName, position)	engine/VisualNovelEngine.js:6858	Detiene character pose animation dentro del flujo correspondiente.
applyPoseClass(charElement, pose)	engine/VisualNovelEngine.js:6872	Marca la pose activa en el elemento (clase "pose-<nombre>") para que el CSS pueda tratar poses concretas de forma distinta. Se usa, por ejemplo, con los primerísimos planos de llanto ("bua"), que en vez de centrarse se pegan a la esquina in

Función y firma	Ubicación	Responsabilidad
playCharacterGlitchSound()	engine/VisualNovelEngine.js:6895	Glitch visual puntual para llamadas o señales comprimidas. La clase se reinicia siempre para que una segunda interrupción vuelva a animarse y se retira al acabar, garantizando que el sprite recupere su forma normal.
triggerCharacterAnimeFall(characterName, position, options = {})	engine/VisualNovelEngine.js:6908	Reacción cómica de anime: anticipación, desplome vertical, golpe, pausa y reincorporación. Se programa sin bloquear el diálogo para que el personaje reaccione mientras se escribe la frase que provoca la caída.
clearCharacterAnimeFall(charElement)	engine/VisualNovelEngine.js:6943	Limpia character anime fall dentro del flujo correspondiente.
triggerCharacterGlitch(characterName, position, duration = 1350)	engine/VisualNovelEngine.js:6955	Dispara character glitch dentro del flujo correspondiente.
triggerCharacterFullGlitch(characterName, position, duration = 1050)	engine/VisualNovelEngine.js:6976	Segundo golpe de interferencia para el cierre de una llamada: afecta al sprite completo sin sustituir el glitch independiente del retrato.
startCharacterGlitchUntilAdvance(characterName, position)	engine/VisualNovelEngine.js:6998	Interferencia persistente ligada a una sola línea de diálogo. No deforma la escala del personaje: el movimiento y la separación RGB los resuelve CSS mediante capas recortadas. nextLine() la retira justo al avanzar.
clearAdvanceBoundCharacterEffects()	engine/VisualNovelEngine.js:7010	Limpia advance bound character effects dentro del flujo correspondiente.
updateCharacterVideo(charElement, videoPath)	engine/VisualNovelEngine.js:7019	Actualiza character video dentro del flujo correspondiente.
resetCharacterSlotElement(charElement, position)	engine/VisualNovelEngine.js:7054	Limpieza integral de un hueco de personaje. Centralizarla evita que un temporizador de salida/glitch o un video de una escena anterior sobreviva a Retroceder, al selector de escenas o a la reutilización del mismo hueco.
hideCharacter(characterName, position, exit = null)	engine/VisualNovelEngine.js:7118	Oculta solo la representación del personaje. Mantiene pose, sprite, posición y ficha en memoria para que el cursor-retrato pueda reutilizar exactamente la pose declarada por showCharacter. removeCharacter es la operación destructiva que vac
removeCharacter(characterName, position, exit = null, suppressLayout = false)	engine/VisualNovelEngine.js:7163	Quita a un personaje de la escena. Se puede indicar: - characterName: quita al personaje (usando la posición rastreada en la que se mostró; si además se da position, solo quita si coincide). - position (sin characterName): vacía directament
removeAllCharacters()	engine/VisualNovelEngine.js:7207	Todo cambio real de fondo inaugura una composición nueva. Vaciar los tres huecos mediante removeCharacter evita que un sprite, una animación o una pose oculta sobrevivan accidentalmente desde el fondo anterior.
focusCharacter(characterName, position)	engine/VisualNovelEngine.js:7216	Enfoca character dentro del flujo correspondiente.
unfocusCharacter(position)	engine/VisualNovelEngine.js:7227	Desenfoca character dentro del flujo correspondiente.
readableNameColor(color)	engine/VisualNovelEngine.js:7238	Devuelve el color del personaje asegurando un mínimo de luminosidad para que el nombre se lea sobre la caja oscura (aclara los colores oscuros mezclándolos hacia el blanco, conservando el tono). Acepta cualquier formato CSS (hex, nombre com
preloadAudio(path)	engine/VisualNovelEngine.js:7280	Preload suave de un path de audio. Sirve para deducir decodificación y URL antes del primer play; la promesa queda en caché y reutiliza esa precarga para siguientes llamadas del mismo source.
_audioPoolKey(soundPath) [interna]	engine/VisualNovelEngine.js:7314	Implementa la operación audio pool key del módulo.
_acquireShortAudioFromPool(poolKey) [interna]	engine/VisualNovelEngine.js:7321	Implementa la operación acquire short audio from pool del módulo.
_releaseShortAudioToPool(audio, poolKey) [interna]	engine/VisualNovelEngine.js:7336	Implementa la operación release short audio to pool del módulo.
_drainAudioShortPool() [interna]	engine/VisualNovelEngine.js:7372	Implementa la operación drain audio short pool del módulo.

Función y firma	Ubicación	Responsabilidad
_snapshotForHistory(value) [interna]	engine/VisualNovelEngine.js:7390	Implementa la operación snapshot for history del módulo.
volFactor(kind)	engine/VisualNovelEngine.js:7405	Factor de volumen del panel de Configuración (0..1, persistente).
applyVolumeSettings()	engine/VisualNovelEngine.js:7412	Reaplicar los ajustes de volumen a todo lo que esté sonando (los fades en curso se dejan terminar con su objetivo antiguo; caso raro y breve).
playSound(soundPath, options = {})	engine/VisualNovelEngine.js:7422	Crea o reutiliza audio, aplica volumen, loop, fundido e identificador de control.
forgetAudio(audio)	engine/VisualNovelEngine.js:7583	Elimina únicamente las referencias que todavía apuntan a este elemento. El audio puede continuar su fade con una referencia local, pero una nueva acción con el mismo ID debe poder crear y arrancar otra pista enseguida.
fadeOutAndStop(audio, ms = 300)	engine/VisualNovelEngine.js:7597	Desvanece el volumen a 0 en `ms` y luego pausa y rebobina. Cancela cualquier fade anterior sobre el mismo audio (evita intervalos compitiendo por el volumen). Con ms<=0, o si ya está pausado, para de inmediato.
stopSound(audioOrId, fadeOut = 0)	engine/VisualNovelEngine.js:7638	Detiene sound dentro del flujo correspondiente.
stopAllSounds()	engine/VisualNovelEngine.js:7654	Detiene y libera todas las instancias de audio para evitar conexiones y reproducción residual.
pauseSound(audioOrId)	engine/VisualNovelEngine.js:7688	Implementa la operación pause sound del módulo.
resumeSound(audioOrId)	engine/VisualNovelEngine.js:7701	Implementa la operación resume sound del módulo.
setVolume(audioOrId, volume)	engine/VisualNovelEngine.js:7714	Establece volume dentro del flujo correspondiente.
wait(ms)	engine/VisualNovelEngine.js:7731	Implementa la operación wait del módulo.
waitForActionClick()	engine/VisualNovelEngine.js:7735	Espera for action click dentro del flujo correspondiente.
normalizeDialogEmotion(value)	engine/VisualNovelEngine.js:7749	Traduce emociones declaradas en una línea (o poses del personaje) a un conjunto pequeño y estable de efectos tipográficos. Los capítulos pueden usar nombres en español o inglés sin acentos ni diferencias de mayúsculas.
resolveDialogEmotion(line, speakerName)	engine/VisualNovelEngine.js:7798	Resuelve dialog emotion dentro del flujo correspondiente.
limitRepeatedDialogEmotion(line, requestedEmotion)	engine/VisualNovelEngine.js:7871	Implementa la operación limit repeated dialog emotion del módulo.
splitTextGraphemes(value = "")	engine/VisualNovelEngine.js:7882	La velocidad y la duración del typewriter se calculan en un único lugar. Así la partida, la vista previa de Configuración y las herramientas de autoría usan exactamente los mismos grafemas y pausas de puntuación.
getTextSpeedMultiplier(line = 1)	engine/VisualNovelEngine.js:7889	Obtiene text speed multiplier dentro del flujo correspondiente.
getTextPunctuationPause(character)	engine/VisualNovelEngine.js:7899	Obtiene text punctuation pause dentro del flujo correspondiente.
getTextCharacterDelay(character, line = 1)	engine/VisualNovelEngine.js:7908	Obtiene text character delay dentro del flujo correspondiente.
getExplicitTextDuration(line = 1)	engine/VisualNovelEngine.js:7914	Obtiene explicit text duration dentro del flujo correspondiente.
calculateTextTiming(line = 1)	engine/VisualNovelEngine.js:7928	Devuelve la estimación completa con el preset seleccionado actualmente. `visibleDurationMs` mide hasta que aparece el último grafema y `durationMs` incluye además la pausa final con la que el motor cierra una línea normal. Si hay textDurationMs se suma a durationMs.
calculateTextDuration(line = 1)	engine/VisualNovelEngine.js:8007	Calcula text duration dentro del flujo correspondiente.
setLineTextDuration(duration)	engine/VisualNovelEngine.js:8014	Acción JSON: { "type": "setTextDuration", "value": 4000 }. Se reinicia al comenzar cada línea para que nunca se filtre a la siguiente.

Función y firma	Ubicación	Responsabilidad
async displayDialog(line) [async]	engine/VisualNovelEngine.js:8022	Pinta hablante y texto, calcula el ritmo del typewriter, gestiona emociones y deja la línea lista para avanzar.
registerCall(scene)	engine/VisualNovelEngine.js:8568	Registrar que se ha completado una llamada según el título de la escena
isCallChoiceCompleted(choice)	engine/VisualNovelEngine.js:8581	Devuelve true si la escena de destino de una llamada ya fue completada
isCallChoice(choice)	engine/VisualNovelEngine.js:8591	Indica si una opción es una llamada a un amigo (destino de escena de llamada)
canMakeRealCall()	engine/VisualNovelEngine.js:8601	Regla del teléfono: Samu solo puede completar una llamada "real" por cada amigo rescatado (más la primera del principio). Es decir, tras llamar a uno, el resto queda "fuera de cobertura" hasta que rescate a alguien. Así, tanto en el Capítulo
async displayChoices(choices) [async]	engine/VisualNovelEngine.js:8605	Construye las opciones, bloquea el avance de fondo y resuelve la elección seleccionada o cancelada.
hayEleccionAbierta()	engine/VisualNovelEngine.js:8655	¿Hay una elección esperando respuesta? La usa el menú de escenas para saber si tiene que abortarla antes de saltar.
abortarEleccion()	engine/VisualNovelEngine.js:8659	Cancela eleccion dentro del flujo correspondiente.
lineConditionMet(cond)	engine/VisualNovelEngine.js:8666	¿Cumple la línea su condición "showIf"? Soporta: { result: "clean" "close" } (resultado del último minijuego), { hasItem: "x" } y { delayAtLeast: n }.
async nextLine() [async]	engine/VisualNovelEngine.js:8685	Ejecuta la siguiente línea, sus acciones, condiciones, diálogo y decisiones; devuelve si queda contenido.
hideDialog()	engine/VisualNovelEngine.js:8884	Oculto dialog dentro del flujo correspondiente.
captureSceneStageState()	engine/VisualNovelEngine.js:8941	una escena, `currentScene` ya apunta a la escena nueva pero el bucle sigue parado esperando el clic del jugador. Si en ese hueco se apilaba tarde, el historial acababa en la escena ANTERIOR y retroceder se volvía loco: creía estar "avanzado"
restoreSceneStageState(snapshot)	engine/VisualNovelEngine.js:8977	Restaura scene stage state dentro del flujo correspondiente.
recordSceneEntry()	engine/VisualNovelEngine.js:9007	Guarda una fotografía del progreso y del escenario para retroceso y menú de escenas.
canRewind()	engine/VisualNovelEngine.js:9033	¿Hay algo a lo que volver? (el botón se oculta si no). Hay dos motivos: queda alguna escena por debajo en la pila, o se ha avanzado dentro de la escena actual y se puede volver a su principio (esto último vale también en la primera escena d
clearStage(options =	engine/VisualNovelEngine.js:9041	Deja el escenario en blanco sin tocar el progreso de la partida. Las cinemáticas preservan el audio para que playVideo gestione su fundido.
restoreSceneSnapshot(foto)	engine/VisualNovelEngine.js:9151	Devuelve el progreso a la foto guardada al entrar en una escena.
replayScene(indiceEscena)	engine/VisualNovelEngine.js:9165	Deja el motor listo para reproducir una escena desde su primera línea. La escena ya está en `sceneHistory` con su foto de progreso, así que se marca como vista para que recordSceneEntry no vuelva a apilarla.
rewindToPreviousScene()	engine/VisualNovelEngine.js:9178	Retrocede al principio de la escena actual o de la anterior, según se haya avanzado ya por esta escena o no (ver el bloque de arriba).
sceneList()	engine/VisualNovelEngine.js:9211	Escenas del capítulo actual, para pintar el menú de selección.
chapterTitle()	engine/VisualNovelEngine.js:9222	Implementa la operación chapter title del módulo.
saltarAEscena(destino)	engine/VisualNovelEngine.js:9238	cambiamos de sitio. El repintado lo hacen las propias acciones de la escena: por eso TODAS declaran su fondo y su música en la primera línea. OJO CON EL NOMBRE: `jumpToScene` YA EXISTE arriba en esta misma clase (es el que atiende la acción
reset()	engine/VisualNovelEngine.js:9277	Devuelve el motor a un estado inicial sin borrar el progreso que debe persistir entre capítulos.

Función y firma	Ubicación	Responsabilidad
updateDebugPanel()	engine/VisualNovelEngine.js:9367	Actualiza debug panel dentro del flujo correspondiente.
goToLine(lineNumber)	engine/VisualNovelEngine.js:9385	Navega a to line dentro del flujo correspondiente.
async playChapterIntro(chapter) [async]	engine/VisualNovelEngine.js:9401	Reproduce o ejecuta chapter intro dentro del flujo correspondiente.
async showChapterEnd(chapterTitle) [async]	engine/VisualNovelEngine.js:9439	Muestra chapter end dentro del flujo correspondiente.

20. Referencia de módulos del motor

ActionInterpreter

Función y firma	Ubicación	Responsabilidad
constructor(engine)	engine/ActionInterpreter.js:303	Inicializa la instancia y sus dependencias internas.
normalize(action)	engine/ActionInterpreter.js:307	Implementa la operación normalize del módulo.
async execute(action) [async]	engine/ActionInterpreter.js:311	Implementa la operación execute del módulo.
async executeObject(action) [async]	engine/ActionInterpreter.js:321	Implementa la operación execute object del módulo.
export function parseAction(action)	engine/ActionInterpreter.js:201	Convierte una cadena DSL en el objeto de acción equivalente y valida sus argumentos.
export function serializeAction(action)	engine/ActionInterpreter.js:248	Convierte un objeto de acción plano a DSL cuando el round-trip puede conservar todos sus datos.

AudioController

Función y firma	Ubicación	Responsabilidad
constructor(engine)	engine/AudioController.js:2	Inicializa la instancia y sus dependencias internas.
play(path, options)	engine/AudioController.js:6	Implementa la operación play del módulo.
stop(id, fadeOut = 0)	engine/AudioController.js:10	Implementa la operación stop del módulo.
stopAll()	engine/AudioController.js:14	Detiene all dentro del flujo correspondiente.
pause(id)	engine/AudioController.js:18	Implementa la operación pause del módulo.
resume(id)	engine/AudioController.js:22	Implementa la operación resume del módulo.
setVolume(id, volume)	engine/AudioController.js:26	Establece volume dentro del flujo correspondiente.
snapshot()	engine/AudioController.js:30	Implementa la operación snapshot del módulo.

DOMRenderer

Función y firma	Ubicación	Responsabilidad
constructor(root = document)	engine/DOMRenderer.js: 2	Inicializa la instancia y sus dependencias internas.
get(id)	engine/DOMRenderer.js: 7	Implementa la operación get del módulo.
getBackgroundElement()	engine/DOMRenderer.js: 15	Obtiene background element dentro del flujo correspondiente.
appendToGame(element)	engine/DOMRenderer.js: 19	Implementa la operación append to game del módulo.
removeAll(selector)	engine/DOMRenderer.js: 26	Retira all dentro del flujo correspondiente.

HistoryManager

Función y firma	Ubicación	Responsabilidad
constructor(engine)	engine/HistoryManager.js:2	Inicializa la instancia y sus dependencias internas.
captureStage()	engine/HistoryManager.js:6	Captura stage dentro del flujo correspondiente.
restoreStage(snapshot)	engine/HistoryManager.js:10	Restaura stage dentro del flujo correspondiente.
rewind()	engine/HistoryManager.js:14	Implementa la operación rewind del módulo.
jump(target)	engine/HistoryManager.js:18	Implementa la operación jump del módulo.
clear()	engine/HistoryManager.js:22	Implementa la operación clear del módulo.

TimeManager

Función y firma	Ubicación	Responsabilidad
constructor(browserWindow = window)	engine/TimeManager.js: 44	Inicializa la instancia y sus dependencias internas.
install()	engine/TimeManager.js: 77	Implementa la operación install del módulo.
invoke(callback, args)	engine/TimeManager.js: 94	Implementa la operación invoke del módulo.
armTimer(record)	engine/TimeManager.js: 98	Programa timer dentro del flujo correspondiente.
createTimer(callback, delay, args, interval)	engine/TimeManager.js: 118	Crea timer dentro del flujo correspondiente.
clearTimer(id)	engine/TimeManager.js: 140	Limpia timer dentro del flujo correspondiente.
armFrame(record)	engine/TimeManager.js: 151	Programa frame dentro del flujo correspondiente.
createFrame(callback)	engine/TimeManager.js: 159	Crea frame dentro del flujo correspondiente.
cancelFrame(id)	engine/TimeManager.js: 167	Cancela frame dentro del flujo correspondiente.
setPaused(value)	engine/TimeManager.js: 177	Establece paused dentro del flujo correspondiente.
isPaused()	engine/TimeManager.js: 203	Comprueba si está paused dentro del flujo correspondiente.
diagnostics()	engine/TimeManager.js: 207	Implementa la operación diagnostics del módulo.
export function installTimeManager(browserWindow = window)	engine/TimeManager.js: 212	Implementa la operación install time manager del módulo.

MinigameBase

Función y firma	Ubicación	Responsabilidad
constructor(options =	minigames/MinigameBase.js:4	Inicializa la instancia y sus dependencias internas.
start()	minigames/MinigameBase.js:11	Implementa la operación start del módulo.
play()	minigames/MinigameBase.js:15	Implementa la operación play del módulo.
listen(target, type, listener, options)	minigames/MinigameBase.js:19	Implementa la operación listen del módulo.
own(cleanup)	minigames/MinigameBase.js:24	Implementa la operación own del módulo.
ownElement(element)	minigames/MinigameBase.js:29	Declara como recurso propio element dentro del flujo correspondiente.
attachOverlay(element)	minigames/MinigameBase.js:34	Asocia overlay dentro del flujo correspondiente.
timeout(callback, delay = 0, ...args)	minigames/MinigameBase.js:47	Implementa la operación timeout del módulo.
animationFrame(callback)	minigames/MinigameBase.js:58	Implementa la operación animation frame del módulo.
pause()	minigames/MinigameBase.js:69	Implementa la operación pause del módulo.
resume()	minigames/MinigameBase.js:75	Implementa la operación resume del módulo.
cleanup()	minigames/MinigameBase.js:80	Implementa la operación cleanup del módulo.
cancel()	minigames/MinigameBase.js:94	Comprueba si se puede cel dentro del flujo correspondiente.
static cleanupAll() [static]	minigames/MinigameBase.js:106	Cancela todas las instancias de minijuego registradas y devuelve cuántas se limpiaron.

MediaRegistry

Función y firma	Ubicación	Responsabilidad
constructor(browserWindow = window)	engine/TimeManager.js:2	Inicializa la instancia y sus dependencias internas.
install()	engine/TimeManager.js:9	Implementa la operación install del módulo.

21. Referencia completa de game.js

Menú, Tools y navegación temprana

Función y firma	Ubicación	Responsabilidad
function setMainMenuVisible(visible)	game.js:93	Establece main menu visible dentro del flujo correspondiente.
function closeToolsOfflineDialog()	game.js:141	Cierra tools offline dialog dentro del flujo correspondiente.
function toolsFailureMessage(result)	game.js:147	Implementa la operación tools failure message del módulo.
function showToolsOfflineDialog(result)	game.js:167	Muestra tools offline dialog dentro del flujo correspondiente.
async function checkEyeToolsHealth() [async]	game.js:180	Implementa la operación check eye tools health del módulo.
async function openEyeTools() [async]	game.js:236	Abre eye tools dentro del flujo correspondiente.
function desbloquearBucle(siguiendoFalta)	game.js:360	Libera de forma segura una espera de diálogo, elección o minijuego para atender navegación global.
function cutsceneEnMarcha()	game.js:411	Los botones de arriba se ven también durante los minijuegos: atascarse en uno era la única forma de quedarse encerrado sin salida que no fuera ganarlo. Al pulsarlos, el motor aborta el minijuego (engine.abortarMinijuego). La excepción es la
function updateRewindButton()	game.js:415	Actualiza rewind button dentro del flujo correspondiente.
function startRewindWatcher()	game.js:422	Inicia rewind watcher dentro del flujo correspondiente.
function stopRewindWatcher()	game.js:431	Detiene rewind watcher dentro del flujo correspondiente.
function updateOptionsButton()	game.js:442	El botón de opciones sigue la misma regla que los otros dos: solo se esconde durante una cutscene, donde Esc es del vídeo.
function updateScenesButton()	game.js:464	El menú de escenas Sí se ofrece durante una elección: es navegación, y si no el jugador se queda encerrado en la pantalla de elección sin poder ir a otra escena (el bucle está esperando respuesta). Al saltar se aborta la elección. Y lo mismo
function cerrarMenuEscenas()	game.js:471	Implementa la operación cerrar menu escenas del módulo.
function abrirMenuEscenas()	game.js:475	Implementa la operación abrir menu escenas del módulo.

Configuración

Función y firma	Ubicación	Responsabilidad
function storedTextSpeed()	game.js:555	Implementa la operación stored text speed del módulo.
function applyTextSpeed(index)	game.js:560	Aplica text speed dentro del flujo correspondiente.
function saveSetting(key, value)	game.js:574	En la app de escritorio el localStorage se pierde en cada arranque (el servidor interno cambia de puerto, y con él de origen), así que el ajuste se copia además a la carpeta de datos de la app, que es quien lo devuelve al abrir. En el navegador
function windowModeActual()	game.js:584	Implementa la operación window mode actual del módulo.
function settingsMarkup()	game.js:595	Los mismos ajustes salen en dos sitios: en Configuración (menú principal) y en el menú de pausa (Esc durante la partida). Se generan y se conectan aquí una sola vez para que no se dupliquen ni se desincronicen. Van repartidos en pestañas (J)
function wireSettings(panel)	game.js:712	Conecta controles de settings dentro del flujo correspondiente.

Galería

Función y firma	Ubicación	Responsabilidad
function showSettingsPanel()	game.js:836	Configuración se abre igual que Capítulos: pantalla completa con el fondo difuminado detrás y el menú principal retirado, no como una cajita encima. De ahí que comparta las clases `chapter-selector*`; lo propio de este panel (ancho, sin lis
function galleryAssetUrl(path)	game.js:869	Implementa la operación gallery asset url del módulo.
function loadGalleryManifest()	game.js:873	Carga gallery manifest dentro del flujo correspondiente.
function attachGalleryLayerBlinks(manifest, layerManifest)	game.js:895	Asocia gallery layer blinks dentro del flujo correspondiente.
function attachGalleryCleanSprites(manifest, cleanManifest)	game.js:907	Asocia gallery clean sprites dentro del flujo correspondiente.
function galleryFocusableElements(scope)	game.js:937	Implementa la operación gallery focusable elements del módulo.
function focusGalleryElement(element)	game.js:943	Enfoca gallery element dentro del flujo correspondiente.
function trapGalleryFocus(event, scope)	game.js:955	Implementa la operación trap gallery focus del módulo.
function closeGalleryPanel()	game.js:974	Cierra gallery panel dentro del flujo correspondiente.
function showGalleryLoadError(error)	game.js:992	Muestra gallery load error dentro del flujo correspondiente.
async function showGalleryPanel() [async]	game.js:1031	Muestra gallery panel dentro del flujo correspondiente.
function renderGalleryPanel(manifest)	game.js:1063	Renderiza gallery panel dentro del flujo correspondiente.

Pausa y entrada

Función y firma	Ubicación	Responsabilidad
function menuPausaAbierto()	game.js:1781	Implementa la operación menu pausa abierto del módulo.
function esPistaMusical(media)	game.js:1788	La pausa congela la partida, pero el menú de Opciones no debe cortar su banda sonora. Se identifica por la carpeta de música para no dejar vivos también efectos en bucle (motores, ambiente, etc.).
function setGamePaused(paused)	game.js:1794	Congela o reanuda reloj virtual, multimedia y contextos de audio de la partida.
function cerrarMenuPausa()	game.js:1852	Implementa la operación cerrar menu pausa del módulo.
function abrirMenuPausa()	game.js:1857	Implementa la operación abrir menu pausa del módulo.
function salirAlMenuPrincipal()	game.js:1894	El bucle de juego casi siempre está parado dentro de waitForClick(), así que no basta con bajar la bandera: hay que despertarlo igual que hacen los botones de retroceder y de escenas. playGame() atiende la petición y sale.
function targetPermiteInputEnPausa(target)	game.js:1904	Implementa la operación target permite input en pausa del módulo.
function bloquearInputDurantePausa(e)	game.js:1908	Implementa la operación bloquear input durante pausa del módulo.
function alternarHUD()	game.js:1914	Implementa la operación alternar hud del módulo.
function hudOculto()	game.js:1921	Implementa la operación hud oculto del módulo.
function mostrarHUD()	game.js:1927	Al terminar la partida el HUD vuelve siempre: dejarlo oculto arrastraría el bloqueo de input al menú o al capítulo siguiente.
function inputBloqueadoPorHUD(target)	game.js:1936	Con el HUD escondido no se ve el bocadillo, así que cualquier input solo serviría para pasar líneas a ciegas: mientras esté oculto, la historia no avanza. Se dejan pasar el menú de pausa y las elecciones (que siguen a la vista) y los miniju
function bloquearInputConHUDOculto(e)	game.js:1943	Implementa la operación bloquear input con hudoculto del módulo.
function puedeAvanzarDialogo()	game.js:1950	Implementa la operación puede avanzar dialogo del módulo.
function avanzarDialogo()	game.js:1959	Implementa la operación avanzar dialogo del módulo.

Opening y multimedia del menú

Función y firma	Ubicación	Responsabilidad
function fitStartupOpeningToViewport()	game.js:2070	Ajusta startup opening to viewport dentro del flujo correspondiente.
function storedMusicVolume()	game.js:2082	Implementa la operación stored music volume del módulo.
function audibleStartupVolume()	game.js:2087	Implementa la operación audible startup volume del módulo.
function showMainMenuAfterOpening()	game.js:2100	Muestra main menu after opening dentro del flujo correspondiente.
function startStartupOpening()	game.js:2118	Inicia startup opening dentro del flujo correspondiente.
function setupStartupSequence()	game.js:2148	Establece up startup sequence dentro del flujo correspondiente.
function menuVideoStackEl()	game.js:2227	Implementa la operación menu video stack el del módulo.
function menuVideoEls()	game.js:2231	Implementa la operación menu video els del módulo.
function applyMenuVideoRate()	game.js:2237	El video va SIEMPRE mudo y ralentizado; su sonido de base se reproduce aparte (ambiente_menu.mp3) para que no se estire ni cambie de tono al frenar el video.
function menuVideoVisible()	game.js:2245	Implementa la operación menu video visible del módulo.
function cancelMenuVideoScheduling()	game.js:2254	Cancela menu video scheduling dentro del flujo correspondiente.
function normalizeMenuVideoLayers(videos = menuVideoEls())	game.js:2262	Normaliza menu video layers dentro del flujo correspondiente.
function finishMenuVideoCrossfade(outgoingIndex, incomingIndex, runId)	game.js:2269	Finaliza menu video crossfade dentro del flujo correspondiente.
async function crossfadeMenuVideo() [async]	game.js:2285	Implementa la operación crossfade menu video del módulo.
function monitorMenuVideoLoop()	game.js:2355	Vigila menu video loop dentro del flujo correspondiente.
function startMenuVideoLoop()	game.js:2376	Inicia menu video loop dentro del flujo correspondiente.
function stopMenuVideoLoop()	game.js:2405	Detiene menu video loop dentro del flujo correspondiente.
function startMenuAmbience()	game.js:2424	DESACTIVADO (jul 2026): el ambiente de base ya no suena en el menú; la capa musical queda en manos del tema (una vez) + menu_chill en bucle. Se conserva la función por si algún día se quiere recuperar.
function playMenuChill()	game.js:2431	Música chill de relevo: entra cuando termina el tema (o al volver al menú) y se aparta cuando el tema vuelve a sonar (botón □). Si el mp3 aún no existe, falla en silencio y no pasa nada.
function playMenuTheme()	game.js:2450	El tema del menú suena UNA sola vez (como una intro); al terminar entra la música chill de fondo. El botón □ permite volver a escucharlo cuando se quiera.
function unlockMenuAudio(e)	game.js:2475	Desbloquea menu audio dentro del flujo correspondiente.

Capítulos y bucle principal

Función y firma	Ubicación	Responsabilidad
function stopMenuMedia()	game.js:2509	Fundir y ocultar el vídeo + sonidos del menú (al empezar a jugar)
function showMenuMedia(playChillOnReturn = true)	game.js:2533	Volver a mostrar el menú con su vídeo y su ambiente (al regresar al menú). El tema NO se relanza solo: para volver a oírlo está el botón  .
async function initGame() [async]	game.js:2545	Implementa la operación init game del módulo.
function ensureAvailableChapters()	game.js:2550	Implementa la operación ensure available chapters del módulo.
async function loadAvailableChapters() [async]	game.js:2566	Carga available chapters dentro del flujo correspondiente.
async function loadAllCharacters() [async]	game.js:2609	Carga all characters dentro del flujo correspondiente.
async function startNewGame() [async]	game.js:2636	Inicia new game dentro del flujo correspondiente.
function releaseChapterTransition(transitionCurtain)	game.js:2661	Implementa la operación release chapter transition del módulo.
async function playChapter(chapterIdentifier, transitionCurtain = null) [async]	game.js:2674	Reproduce o ejecuta chapter dentro del flujo correspondiente.
async function playGame() [async]	game.js:2714	Mantiene el bucle principal y atiende salida, retroceso, salto de escena y avance de líneas.
function waitForClick()	game.js:2777	Espera for click dentro del flujo correspondiente.
function volverAlMenuPrincipal()	game.js:2808	Abandona la partida en curso y deja el menú principal como al arrancar. No pasa por endGame(): ahí hay pantalla de fin de capítulo y encadenado con el siguiente, y esto es una salida seca a mitad de capítulo.
async function endGame() [async]	game.js:2826	Implementa la operación end game del módulo.
async function checkChapterExists(chapterName) [async]	game.js:2872	Implementa la operación check chapter exists del módulo.
async function showContinueOptions(nextChapterId, transitionCurtain) [async]	game.js:2880	Muestra continue options dentro del flujo correspondiente.
async function loadGame() [async]	game.js:2916	Carga game dentro del flujo correspondiente.
function showChapterSelector()	game.js:2938	Muestra chapter selector dentro del flujo correspondiente.

Guardado y depuración

Función y firma	Ubicación	Responsabilidad
async function startChapterFromSelector(chapterId) [async]	game.js:2991	Inicia chapter from selector dentro del flujo correspondiente.
function saveGame()	game.js:3018	Guarda game dentro del flujo correspondiente.
function initDebugMode()	game.js:3030	==== Debug Mode =====
function toggleDebugPanel()	game.js:3082	Alterna debug panel dentro del flujo correspondiente.

22. Recetas rápidas

Añadir una línea con personaje

```
"actions": ["show nexo right happy"],
"character": "Nexo",
"text": "Enlace establecido.",
"emotion": "joy"
```

Cambiar de ambiente con fundido

```
"actions": [
  "sceneStart assets/images/backgrounds/chapterX/noche.webp duration=450",
  "sound assets/audio/music/chapterX/noche.mp3 id=bg_music loop=true fadeIn=900"
]
```

Lanzar un minijuego

```
"actions": [  
  "minigame battle enemy=amalgama enemyHp=840 retryOnDefeat=true useInventory=false"  
]
```

Conservar una acción compleja

```
"actions": [{  
  "type": "animateCharacter",  
  "character": "3c",  
  "position": "left",  
  "poses": ["neutral", "surprised", "neutral"],  
  "frameMs": 180  
}]
```

Añadir una ruta

```
"choices": [  
  {"text": "Seguir", "nextScene": "Escena 3: Camino"},  
  {"text": "Llamar", "nextChapter": "chapter_ruta"}  
]
```

23. Glosario y fuentes internas

Término	Definición
Acción	Orden ejecutable previa o posterior a una línea.
DSL	Sintaxis textual compacta que representa objetos de acción planos.
Round-trip	Serializar y volver a parsear sin cambiar ningún dato.
Bundle	engine.bundle.js generado desde los módulos ES para carga clásica.
Snapshot	Fotografía de escena usada por historial y retroceso.
Overlay	Capa DOM temporal de minijuego, vídeo, modal o pausa.
RAF	requestAnimationFrame; bucle de actualización sincronizado con pintado.
Canon	Contenido narrativo vigente; 3C y Nexa ocupan los roles activos documentados.
Workbench	Espejo versionado de masters y originales, excluido del paquete Electron.

23.1 Fuentes verificadas

- DOCUMENTACION.md y LEER_PRIMERO.md.
- engine/VisualNovelEngine.js y los módulos de engine/.
- game.js, index.html y styles.css.
- chapters/chapter0.json a chapter6.json y characters/*.json.
- minigames/MinigameBase.js y los módulos externos de minijuegos.
- package.json, scripts de validación y tests/engine/.
- Capturas del servidor local http://localhost:8000 realizadas el 8 de agosto de 2026.

Fin de la edición

Manual generado desde 7 capítulos, 64 escenas, 921 líneas y 1392 acciones. Regenera el PDF después de cambios de API, esquema, controles o contenido visible.